Manual técnico

Sistema de Gestión Documental

Versión 1.0 · 24 de octubre de 2026 · Angular 21 · Express + Mongoose · MongoDB 7 (replica set) · GridFS

Pág.	Contenido	Para qué sirve
1	Visión general y tecnologías	Qué es el sistema y con qué está construido
2	Arquitectura lógica	Capas, módulos y flujo ruta → controlador → servicio
3	Estructura del repositorio	Dónde está cada cosa
4	Configuración y variables de entorno	Qué configurar y qué valores usar
5	Instalación y ejecución	Con Docker y sin Docker
6	Seguridad y permisos	JWT, roles, S3 y matriz de permisos
7–8	Operaciones del sistema (patrones de acceso)	Cada operación con colecciones, reglas e índices
9	Transacciones y consistencia	Carga, edición y eliminación con GridFS
10	Reportes y agregaciones	R1–R11 y su pipeline
11	Scripts y comandos	npm, Docker y scripts de la carpeta scripts/
12	Operación y solución de problemas	Verificación, datos de prueba y errores comunes

1. Visión general y tecnologías

Aplicación web para gestionar documentos de texto (.txt y .md) con permisos, papelera, historial y reportes administrativos.

Qué hace el sistema

- Autenticación y gestión de usuarios (solo el administrador crea, activa o desactiva usuarios y ajusta cuotas).
- Explorador unificado de carpetas y archivos con permisos heredados.
- Subida, edición de contenido, renombrado y movimiento con validación UTF-8 y de extensión `.txt` / `.md`.
- Papelera con restauración por grupo o ítem y eliminación definitiva.
- Compartición directa (lectura / edición), «compartidos por mí» y «compartidos conmigo».
- Historial de actividad según rol y propiedad; búsqueda unificada con filtros por tipo y fecha.
- Vista previa en línea para TXT y MD (sin registrar actividad).
- Reportes administrativos con agregaciones MongoDB.

Alcance y límites

- Solo formatos `.txt` y `.md`, contenido UTF-8.
- No hay límite individual de tamaño: el límite efectivo es la cuota del usuario (o ninguno si es ilimitada).
- El permiso `comment` queda fuera del alcance (D-42).
- No hay autorregistro; los usuarios no se eliminan físicamente.
- El rol administrador no concede acceso al contenido de archivos ajenos (D-68).

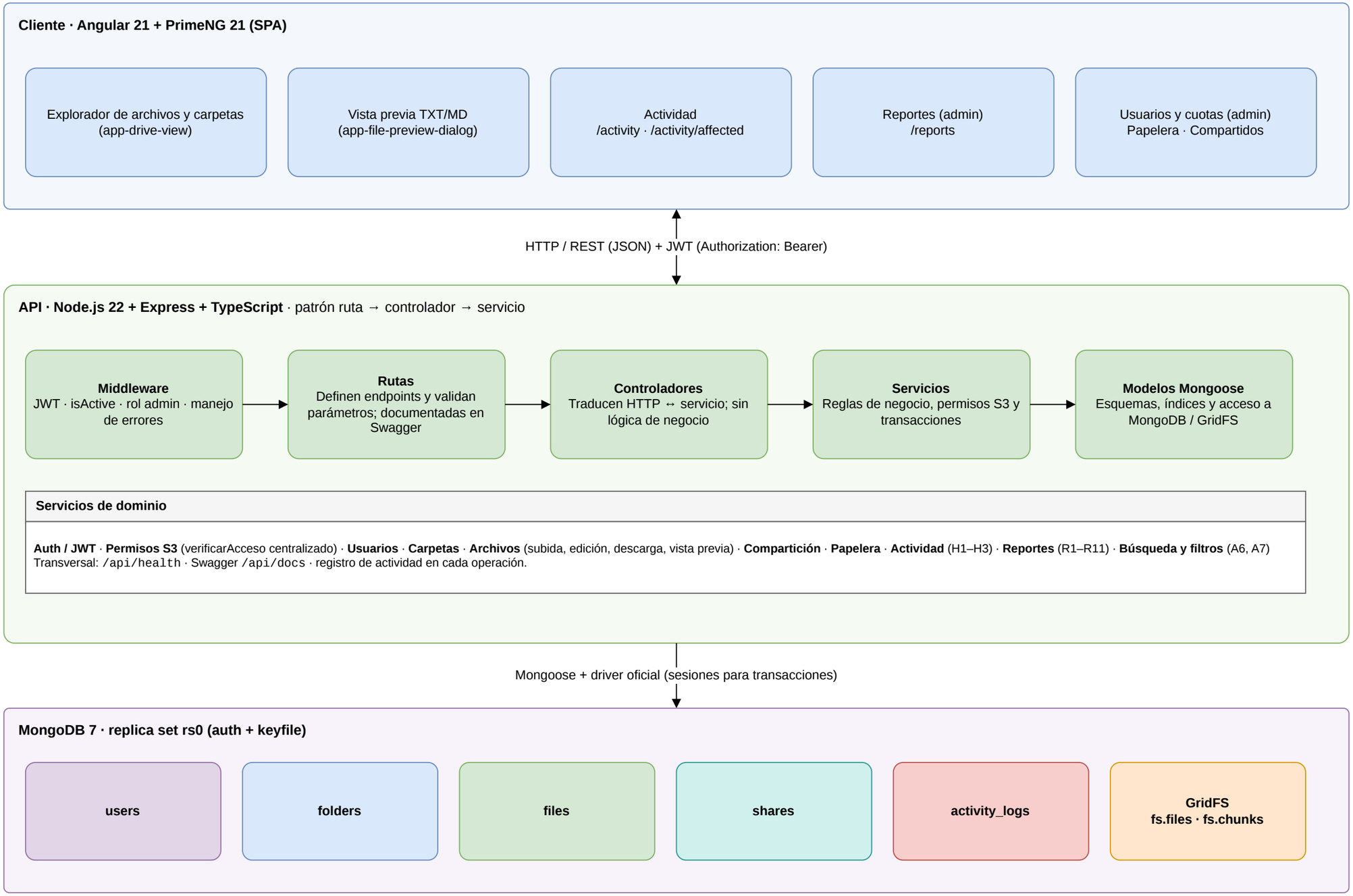
Capa	Tecnología	Detalle
Frontend	Angular 21, PrimeNG 21, TypeScript	Componentes standalone, signals, `ng serve`
Backend	Node.js 22, Express, Mongoose, TypeScript	Patrón ruta → controlador → servicio; nodemon en desarrollo
Base de datos	MongoDB 7	Replica set `rs0` (transacciones), autenticación y keyfile
Contenido	GridFS	`fs.files` / `fs.chunks`
Autenticación	bcrypt + JWT	Token con `sub` y `role`; 8 h por defecto
Documentación API	Swagger	`/api/docs`
Contenedores	Docker Compose	Servicios mongo, mongo-init, api y client

Puertos y URLs por defecto

Cliente (Angular dev): http://localhost:4200
API: http://localhost:3000 · Health: `/api/health` · Swagger: `/api/docs`
MongoDB: localhost:27017 (replica set `rs0`, usuario `admin` en desarrollo)

2. Arquitectura lógica

Angular consume una API REST protegida con JWT; la API accede a MongoDB y GridFS, que solo son accesibles desde el servidor.



3. Estructura del repositorio

Carpetas y archivos principales en la raíz del proyecto.

Ruta	Contenido
`docker-compose.yml`	Servicios mongo, mongo-init, api y client; volúmenes `mongo-data`, `mongo-key`, `api-node-modules`, `client-node-modules`.
`.env` / `.env.example`	Variables de entorno de la raíz (Docker y fuente de valores para ejecución local). `.env` no se versiona; `.env.example` es la plantilla.
`api/`	API Express + TypeScript: `src/` (código), `package.json`, `nodemon.json`, `tsconfig.json`, `Dockerfile` (node:22-alpine, `nodemon --legacy-watch src/server.ts`). En ejecución local lee `api/.env`.
`api/src/`	Rutas, controladores, servicios y modelos; punto de entrada `server.ts`. Scripts de base de datos: índices, seeds, reset, limpieza de GridFS y diagrama.
`client/`	Angular 21: `src/` (páginas, `core/` servicios HTTP, `models/`, `shared/`), `public/`, `angular.json`, `tsconfig*.json`, `Dockerfile` (node:22-alpine, `npm run start` con `--host 0.0.0.0 --port 4200 --poll 2000`).
`docs/`	`decisions.md` (decisiones técnicas y uso de funcionalidades MongoDB), `erd.md` (esquema e índices, generado con `npm run db:diagram`).
`scripts/`	`run-docker.ps1`, `setup-env.sh` / `setup-env.ps1` (generan `api/.env`), `local-mongo.sh` / `local-mongo.ps1` (MongoDB local con auth + keyfile + replica set).
`mongo-data/local-auth/`	Datos, keyfile y log del MongoDB local sin Docker (se crea con `local-mongo start`; no se versiona).

4. Configuración y variables de entorno

Docker usa un único .env en la raíz. Sin Docker, scripts/setup-env.* genera api/.env a partir de él.

Variable	Valor en desarrollo	Usada por	Descripción
`NODE_ENV`	`development`	api	Modo de ejecución.
`API_PORT` / `PORT`	`3000`	compose / api	`API_PORT` publica el puerto en Docker; `PORT` es el que escucha la API.
`CLIENT_PORT`	`4200`	compose	Puerto publicado del cliente.
`CLIENT_URL`	`http://localhost:4200`	api	Origen del cliente (CORS).
`MONGO_PORT`	`27017`	compose / scripts	Puerto de MongoDB.
`MONGO_ROOT_USER`	`admin`	mongo / scripts	Usuario root de MongoDB.
`MONGO_ROOT_PASSWORD`	`admin123`	mongo / scripts	Contraseña root. **Cambiar fuera de desarrollo.**
`MONGO_DB`	`gestion_documental`	api / mongo	Nombre de la base de datos.
`MONGO_REPLICA_SET`	`rs0`	mongo / api	Nombre del replica set (habilita transacciones y change streams).
`MONGO_URI`	ver abajo	api	Cadena de conexión. En Docker se sobrescribe con host `mongo`; en local usa `127.0.0.1`.
`JWT_SECRET`	`dev-secret-…`	api	Clave de firma del JWT. **Obligatorio cambiarla en producción.**
`JWT_EXPIRES_IN`	`8h`	api	Duración del token.
`ADMIN_EMAIL` / `ADMIN_PASSWORD`	`admin@docsdrive.local` / `Admin123*`	api (seed)	Primer administrador creado por `npm run db:seed-admin`.

Cadenas de conexión

Docker (api → mongo): `mongodb://admin:admin123@mongo:27017/gestion_documental?authSource=admin&replicaSet=rs0`
Local (sin Docker): `mongodb://admin:admin123@127.0.0.1:27017/gestion_documental?authSource=admin&replicaSet=rs0&directConnection=true`
Si la contraseña tiene caracteres especiales (`@ / : %`), deben codificarse en la URI; `setup-env.*` lo hace automáticamente.

5. Instalación y ejecución

Dos formas de iniciar el sistema. Ambas dejan Cliente en :4200, API en :3000 y MongoDB en :27017.

Opción A · Con Docker

Requisitos

Docker Desktop (o Docker Engine) con Docker Compose.

1 · Preparar el .env

Copiar `.env.example` a `.env` en la raíz y ajustar valores si hace falta.

2 · Levantar los servicios

```
docker compose up --build -d
```
o en Windows: `powershell -ExecutionPolicy Bypass -File .\scripts\run-docker.ps1`

3 · Verificar

`docker compose ps` — mongo, api y client deben estar en ejecución; mongo-init termina solo.
Abrir `/api/health` y http://localhost:4200.

4 · Crear datos

```
docker compose exec api npm run db:seed-admin
```
Opcional: `docker compose exec api npm run db:reset` y … `db:seed-minimal`.

5 · Detener

`docker compose down` · `docker compose down -v` borra también los datos.

Opción B · Sin Docker

Requisitos

Node.js 22, MongoDB Server 7 y `mongosh`, todos en el PATH.
Windows: PowerShell 5.1+. Linux/macOS: bash.

1 · Generar api/.env

```
bash scripts/setup-env.sh
```
Windows: `powershell -ExecutionPolicy Bypass -File .\scripts\setup-env.ps1`

2 · Iniciar MongoDB (auth + keyfile + replica set)

`bash scripts/local-mongo.sh start` (Windows: `local-mongo.ps1 start`)
Primer arranque: crea el usuario root; luego inicia con `--replSet rs0 --keyFile` y ejecuta `rs.initiate`.

3 · API y datos iniciales

```
cd api && npm install
npm run db:seed-admin
npm run dev
```

4 · Cliente

```
cd client && npm install && npm start
```

5 · Detener

`Ctrl+C` en API y cliente; `local-mongo stop`. `local-mongo reset` borra los datos locales.

6. Seguridad, roles y permisos

El servidor decide siempre: el cliente nunca determina su identidad ni sus permisos (D-04).

Autenticación

- Contraseñas con **bcrypt**; nunca se devuelve `passwordHash`.
- Login (U1) busca por `emailNormalized` y emite un **JWT** con `sub` y `role` (8 h por defecto).
- El id del usuario sale del `sub` del token, no del cuerpo de la petición.
- **Cada petición protegida** vuelve a validar `users.isActive` (D-50): desactivar a alguien corta su acceso de inmediato.

Autorización S3

- Una sola función centralizada decide el acceso a archivos y carpetas.
- Cortocircuito si el usuario es propietario.
- Si no, consulta `shares` con `resourceId ∈ [recurso, carpeta, …ancestors]` y `sharedWithId`; gana el permiso más alto (`edit` > `read`).
- Un recurso con `isDeleted: true` solo es visible para su propietario en la papelera.
- Los permisos heredados **no se almacenan**: se calculan.

Validación de entradas

- Extensión `.txt` / `.md` y MIME permitido; `category` se deriva de la extensión.
- UTF-8 válido con `TextDecoder` en modo `fatal` y en streaming.
- El texto de búsqueda se **escapa** antes de construir el regex.
- CORS limitado al origen `CLIENT_URL`.
- Secretos solo por variables de entorno; `.env` fuera del repositorio.

Operación	Lectura	Edición	Propietario	Admin (rol)
Listar, descargar, vista previa	Sí	Sí	Sí	No
Editar contenido, renombrar	No	Sí	Sí	No
Crear carpeta / subir en carpeta ajena	No	Sí (en esa carpeta)	Sí	No
Mover archivo (solo dentro del árbol de un mismo propietario)	No	No	Sí	No
Mover carpeta	No	No	Sí	No
Compartir, cambiar o retirar permisos	No	No	Sí	No
Restaurar y eliminar definitivamente	No	No	Sí	No
Crear/activar/desactivar usuarios, ajustar cuotas	No	No	No	Sí
Reportes y actividad global (solo metadatos)	No	No	No	Sí

7. Operaciones del sistema · archivos y carpetas

Resumen de los patrones de acceso (A, C). Detalle completo en Análisis.md, sección 2.

ID	Operación	Colecciones	Reglas clave	Índice principal
A1/C2	Listar contenido de carpeta	folders, files, shares	Paginado, ordenado por `nameNormalized`; recursos propios y compartidos según S3.	`files {folderId, isDeleted, nameNormalized}`
A2	Subir archivo	files, GridFS, users, shares, logs	Requiere `edit` sobre la carpeta. UTF-8 en streaming; cuota reservada de forma atómica en la transacción final. Carpeta ajena → share automático para su dueño.	`users _id`
A3	Descargar	files, GridFS, logs	S3; registra `DOWNLOAD`.	`_id`
A4	Renombrar archivo	files, logs	`edit` o propietario; conserva la extensión; duplicados por `nameNormalized`.	`{folderId, nameNormalized}` único parcial
A5	Mover archivo	files, folders, shares	Solo el propietario; origen y destino con el mismo propietario; solo cambia `folderId` y `updatedAt`.	`{folderId, nameNormalized}` único parcial
A6	Buscar por nombre	files, folders, shares	`GET /api/search`: `q`, `scope` (own/shared/all); regex escapado sobre `nameNormalized`; se fusionan resultados en Node y se pagina.	`files {ownerId, isDeleted, nameNormalized}`
A7	Filtrar por tipo y fecha	files, shares	`GET /api/files/filter`: `category` (TXT/MD), `from`/`to` sobre `createdAt`, `scope`; orden descendente.	`{ownerId, isDeleted, category, createdAt}`
A8	Editar contenido	files, GridFS, users, logs	`edit` o propietario; GridFS nuevo; transacción con delta de cuota; luego se borra el objeto anterior.	`_id`
A9	Metadatos y vista previa	files, shares, GridFS	`GET /api/files/:id/preview` en línea, solo TXT/MD, **sin registrar `VIEW`**.	`_id`
C1	Crear carpeta	folders, shares, logs	`edit` sobre el padre; `ownerId = padre.ownerId`, `createdBy = usuario`; `ancestors = [...padre.ancestors, padre._id]`.	`{parentId, nameNormalized}` único parcial
C3	Mover carpeta	folders, files, shares, logs	Solo el propietario; sin ciclos; actualiza el subárbol en transacción (`updateMany`).	`{ancestors, isDeleted}` multikey
C4	Renombrar carpeta	folders, logs	`edit`; la raíz no se renombra; duplicados por `parentId + nameNormalized`.	`{parentId, nameNormalized}`
C5	Breadcrumb	folders, shares	Solo ancestros autorizados; nunca revela nombres sin acceso.	`_id`

8. Operaciones del sistema · compartición, papelera, usuarios e historial

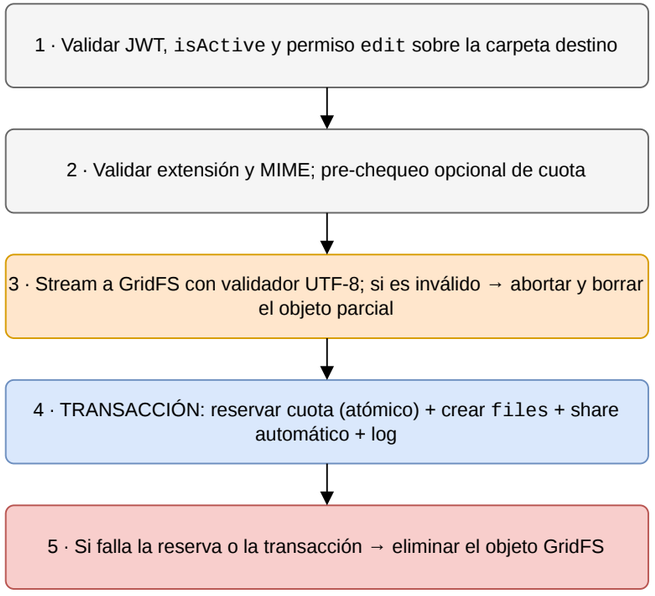
Patrones S, P, U y H.

ID	Operación	Colecciones	Reglas clave	Índice principal
S1	Compartir un recurso	shares, users, logs	Solo el propietario; no se comparten raíces; destino activo y distinto; `upsert`; permisos `read` / `edit`.	`{resourceId, sharedWithId}` único
S2	Cambiar o retirar permiso	shares, logs	Solo shares directos. Retirar el de una carpeta no borra shares directos de sus descendientes.	`{resourceId, sharedWithId}` único
S3	Verificar acceso	files/folders, shares	Propietario cortocircuita; directo + heredado; `edit` > `read`; admin no concede acceso.	`{resourceId, sharedWithId}` único
S4	Compartidos conmigo	shares, files, folders	Origen: shares directos; descarta eliminados **antes** de paginar.	`{sharedWithId, createdAt}`
S5	Compartidos por mí	shares, files, folders	Solo `origin: "manual"`; los automáticos no inflan los reportes.	`{ownerId, origin, createdAt}`
P1	Enviar a papelera	folders, files, logs	`deletionId`; `isDeletionRoot` solo en lo solicitado; una carpeta no puede contener recursos de otros; la cuota no cambia; shares se conservan.	`{ancestors, isDeleted}`
P2	Listar papelera	files, folders	Solo raíces de eliminación del propietario, por `deletedAt` desc.	`{ownerId, deletedAt}` parcial
P3	Restaurar	files, folders, logs	Solo propietario; si el padre no existe o el nombre está ocupado se pide otra ubicación/nombre. **No** verifica cuota (redundante con D-24).	`{deletionId}` parcial
P4	Eliminar definitivamente	files, folders, GridFS, users, shares, logs	Transacción de metadata, shares y cuota (se libera a `file.ownerId`); luego GridFS; los logs se conservan.	`{deletionId}` parcial
U1	Login	users	bcrypt + JWT; busca por `emailNormalized`; usuario activo.	`{emailNormalized}` único
U2–U3	Listar / activar-desactivar usuarios	users, logs	Solo admin; sin eliminación física; no se comparte con inactivos.	`{isActive, role}`
U4	Espacio usado	users, files	Usado, cuota y disponible (`null` si ilimitada); reconciliación contra `files.size`.	prefijo `ownerId`
U5–U6	Crear usuario / ajustar cuota	users, folders, logs	Solo admin; raíz creada en la misma transacción; bajar la cuota bajo `usedBytes` solo bloquea nuevas cargas.	`{ownerId}` único parcial raíz
H1	Registrar actividad	activity_logs	`success` y `error`; un log por carpeta y por cada recurso afectado; completa `affectedUserIds`.	—
H2	Historial de un recurso	activity_logs	Usuario: acciones propias + las del propietario del recurso (`resourceSnapshot.ownerId` si fue eliminado). Admin: global.	`{resourceType, resourceId, timestamp}`
H3	Actividad que me afecta	activity_logs	`affectedUserIds = usuario`, `status: success`, excluye acciones propias.	`{affectedUserIds, timestamp}`

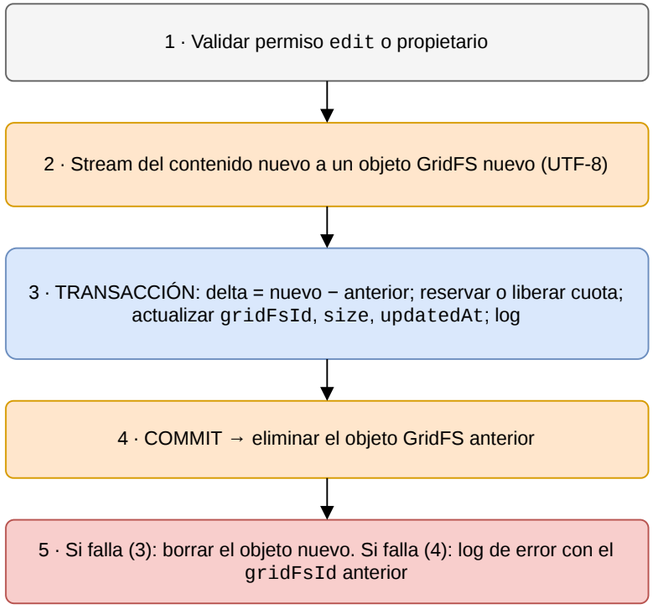
9. Transacciones y consistencia con GridFS

GridFS no participa en transacciones: el contenido y la metadata se confirman en fases, con compensación si algo falla.

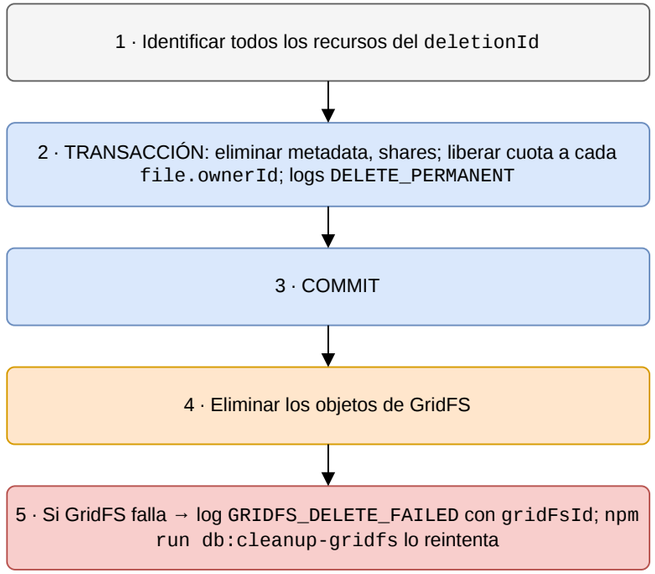
Carga de archivo (A2)



Edición de contenido (A8)



Eliminación definitiva (P4)



Leyenda: Validación · GridFS · Transacción MongoDB · Compensación / error

Por qué dos fases: una transacción de MongoDB no puede deshacer una escritura en GridFS. Se escribe primero el contenido (reversible borrando el objeto) y después se confirma la metadata; en los borrados ocurre al revés para no dejar metadata apuntando a contenido inexistente.

10. Reportes y agregaciones MongoDB

Solo administrador (D-69) · solo status: "success" (D-55) · etapas conceptuales según Análisis v2; el código de api/src es la referencia final.

ID	Reporte	Colección	Etapas del pipeline	Índice
R1	Archivos agrupados por tipo	files	`$match {isDeleted:false}` → `$group` por `category` (count, suma de `size`)	`{isDeleted, createdAt}` (prefijo)
R2	Archivos en un rango de fechas	files	`$match {isDeleted:false, createdAt:{$gte,$lte}}` → `$sort createdAt:-1` → paginación	`{isDeleted, createdAt}`
R3	Archivos con más operaciones	activity_logs	`$match {status:success, resourceType:file}` → `$group` por `resourceId` (count, último) → `$sort` → `$limit` → resolver archivo o `resourceSnapshot`	`{resourceType, resourceId, timestamp}`
R4	Top 10 usuarios por archivos	files	`$match {isDeleted:false}` → `$group` por `ownerId` → `$sort` → `$limit 10` → `$lookup users`	prefijo `ownerId`
R5	Usuarios con más recursos compartidos	shares	`$group` por `ownerId` contando recursos distintos y por `origin` (manual + auto) → `$sort` → `$limit` → `$lookup users`	`{ownerId, origin, createdAt}`
R6	Archivos compartidos con más usuarios	shares	`$match {origin:manual}` → `$group` por `resourceId` (usuarios distintos, último) → `$sort` → `$limit` → `$lookup files/users`	`{resourceId, sharedWithId}`
R7	Recursos compartidos por un usuario	shares	`$match {ownerId}` → `$group` por `resourceId` (destinatarios, orígenes, último) → `$lookup` → paginación (manual y auto)	`{ownerId, origin, createdAt}`
R8	Recursos recibidos por un usuario	shares	`$match {sharedWithId}` → `$lookup` recurso y propietario → `$sort createdAt:-1` → paginación	`{sharedWithId, createdAt}`
R9	Historial de un usuario en un rango	activity_logs	`$match {userId, status:success, timestamp:{$gte,$lte}}` → `$sort timestamp:-1` → paginación	`{userId, timestamp}`
R10	Operaciones por tipo de acción	activity_logs	`$match {status:success}` → `$group` por `action` (count) → `$sort count:-1`	`{action, status, timestamp}`
R11	Operaciones por período	activity_logs	`$match` éxito + rango → `$group` por `$dateTrunc` (`day`/`week`/`month`, zona `America/Guatemala`) → `$sort`	`{timestamp}`

Pestaña en la UI (`/reports`)	Reportes que muestra
Archivos y almacenamiento	R1 archivos por tipo · R2 archivos en rango de fechas · R3 archivos con más operaciones
Usuarios	R4 top 10 por archivos · R5 usuarios con más recursos compartidos
Compartición	R6 archivos compartidos con más usuarios · R7 compartidos por un usuario · R8 recibidos por un usuario
Actividad	R9 historial de un usuario · R10 operaciones por tipo de acción (cubre «principales acciones») · R11 operaciones por período

R12 se omitió (confirmado) por solaparse con R10: agrupar por `action` ordenado descendente ya entrega las principales acciones. Cada pestaña carga sus datos bajo demanda al abrirse.

11. Scripts y comandos

Todos los comandos npm de base de datos se ejecutan desde api/ con MongoDB iniciado y api/.env disponible (o dentro del contenedor con docker compose exec api).

Comando (api/)	Qué hace
`npm run dev`	Inicia la API con nodemon (recarga al cambiar código).
`npm run typecheck`	Verificación TypeScript. Equivalente: `npx tsc --noEmit`.
`npm run db:indexes`	Garantiza (crea si faltan) todos los índices del modelo.
`npm run db:seed-admin`	Crea o asegura el administrador con `ADMIN_EMAIL` y `ADMIN_PASSWORD`.
`npm run db:seed-minimal`	Datos mínimos de prueba: usuarios, carpetas, archivos en GridFS, un share y logs de actividad.
`npm run db:reset`	Borra los datos y reinicia la base. **Destructivo.**
`npm run db:cleanup-gridfs`	Reintenta los borrados de GridFS que fallaron (usa los logs `GRIDFS_DELETE_FAILED`). Idempotente.
`npm run db:diagram`	Actualiza `docs/erd.md`.

Comando (client/)	Qué hace
`npm start`	Servidor de desarrollo de Angular en el puerto 4200.
`npx ng build`	Compila el cliente (verificación tras cambios).

Docker / scripts/	Qué hace
`docker compose up --build -d`	Construye y levanta mongo, mongo-init, api y client.
`docker compose ps` / `logs -f api`	Estado y logs de los servicios.
`docker compose down` / `down -v`	Detiene; con `-v` elimina también los volúmenes de datos.
`scripts/run-docker.ps1`	Valida el `.env` de la raíz y lo pasa a Compose (Windows).
`scripts/setup-env.sh` / `.ps1`	Genera `api/.env` desde el `.env` de la raíz (`--force` / `-Force` para regenerar).
`scripts/local-mongo.sh` / `.ps1`	`start` · `stop` · `status` · `reset` del MongoDB local con auth, keyfile y replica set.

12. Operación, datos de prueba y solución de problemas

Verificación rápida

- `GET /api/health` responde OK (healthcheck del contenedor api).
- Swagger: `/api/docs`.
- Replica set: `mongosh -u admin -p … --authenticationDatabase admin --eval "rs.status().ok"` → 1.
- Tras cada cambio: `npx tsc --noEmit` en api y `npx ng build` en client.

Usuario (seed mínimo)	Rol y cuota
`admin@docsdrive.local` / `Admin123*`	Administrador · ilimitada
`juan@docsdrive.local` / `User123*`	Usuario · 100 MB
`maria@docsdrive.local` / `User123*`	Usuario · ilimitada
`carlos@docsdrive.local` / `User123*`	Usuario · 50 KB

Incluye carpetas, archivos `.txt/.md` en GridFS, un share (Juan → María sobre `README.md`, lectura) y logs de actividad. **Cambiar estas credenciales fuera de desarrollo.**

Síntoma	Causa y solución
`docker` no se reconoce	Instalar Docker Desktop y reiniciar la terminal o Windows.
Mongo: `not primary`	El replica set no está iniciado o aún no es PRIMARY. Docker: revisar `sgd-mongo-init`. Local: `local-mongo status` y `mongod.log`.
`Authentication failed`	Credenciales distintas a las del primer arranque. Local: `local-mongo reset`. Docker: `down -v`.
`Server selection timed out`	Host, puerto o `replicaSet` incorrectos en `MONGO_URI`.
Puerto ocupado (`EADDRINUSE`)	Cerrar el proceso que usa 3000, 4200 o 27017, o cambiar el puerto en `.env`.
CORS desde el cliente	`CLIENT_URL` debe coincidir exactamente con la URL del cliente.
Cuota excedida al subir	Subir `quotaBytes` (U6) o liberar espacio con eliminación definitiva; la papelera no libera cuota.
Contenido rechazado	El archivo no es `.txt/.md` o no es UTF-8 válido.
Objetos GridFS huérfanos	Ejecutar `npm run db:cleanup-gridfs`.
Cambios no se reflejan en Docker	Los volúmenes montan `src`; si cambiaste `package.json`, reconstruir con `up --build`.